

Dokumentacja wstępna

- Mikołaj Garbowski
- Mariusz Pakulski
- Paweł Łasica

Przeprowadzimy eksperymenty, w których porównamy skuteczność różnych strategii agenta generującego zapytania SQL.

Wybrany zbiór danych

Do oceny jakości zaimplementowanego agenta wykorzystamy zbiór danych Spider [1]. Ten zbiór zawiera problemy, które dobrze odzwierciedlają realistyczne problemy analityczne.

Innym popularnym zbiorem do oceny jakości jest WikiSQL [2], natomiast zawarte w nim przykłady są raczej proste (obejmują pojedyncze tabele).

Kolejnym potencjalnym wyborem może być bardzo obszerny zbiór danych BIRD [3], natomiast my decydujemy się na nieco mniejszy i bardzo popularny Spider [4].

Wybrane miary jakości

- Execution accuracy [4] - Określa zgodność wyników zapytania wygenerowanego przez model z wynikami zapytania referencyjnego. Zapytanie uznaje się za poprawne jeśli po wykonaniu na bazie danych zwraca taki sam rezultat jak zapytanie wzorcowe.
- Exact set match accuracy [4] - Polega na porównaniu struktury zapytania SQL wygenerowanego przez model z zapytaniem referencyjnym. Zapytanie uznaje się za poprawne, jeżeli wszystkie jego komponenty, takie jak SELECT, WHERE, GROUP BY czy ORDER BY, odpowiadają komponentom zapytania wzorcowego.
- Execution Accuracy per SQL Hardness [1] - Dzieli zapytania SQL według poziomu złożoności na cztery kategorie:
 - easy,
 - medium,
 - hard,
 - extra hard.

Klasyfikacja ta opiera się na analizie struktury zapytania SQL, w szczególności na obecności:

- złączeń tabel (JOIN),
- grupowania danych (GROUP BY),
- zapytań zagnieżdżonych,
- operacji zbiorowych,

- liczby warunków i agregacji.
- Component matching [1], [5] - Umożliwia analizę poprawności poszczególnych komponentów zapytania SQL takich jak:
 - SELECT,
 - WHERE,
 - GROUP BY,
 - ORDER BY,
 - JOIN,
 - nested SQL.

Wybrane narzędzia

Platforma

Za platformę roboczą przyjmujemy środowisko uruchomieniowe Google Colab. Dostarcza ona możliwość korzystania z dedykowanych kart TPU/GPU oraz uruchamiania notatników Jupyter. W przypadku chęci rozwoju projektu o GUI dopuszczamy możliwość skonteneryzowania aplikacji i uruchamiania jej lokalnie.

Główne narzędzia LLM

- langchain/langgraph [6], [7] - biblioteki dostarczające rozwiązania dotyczące budowania promptów, agentów oraz zdolności rozumowania
- transformers - biblioteka ułatwiająca pracę z wielkimi modelami językowymi
- pytorch - biblioteka ogólnego przeznaczenia do działań głębokiego uczenia oraz tensorów

Baza danych

SQLite - uruchamiania lokalnie relacyjna baza danych. Jest to typowy wybór przy tego typu zadaniach, używana w popularnych zbiorach danych [1], [2], [3].

Porównywane strategie agenta

- Plan and Solve [8] - polega na rozdzieleniu procesu rozwiązywania problemu na dwa etapy - utworzenie planu rozwiązania i wykonanie zaplanowanych kroków. Model językowy najpierw generuje strukturę działania opisującą sposób rozwiązania zadania, a następnie realizuje kolejne kroki prowadzące do wygenerowania zapytania SQL.
- Chain-of-thought [9] - polega na generowaniu rozwiązania poprzez sekwencję kroków rozumowania, bez bezpośredniej interakcji z narzędziami lub środowiskiem wykonawczym w trakcie procesu wnioskowania. Model językowy analizuje zapytanie użytkownika i generuje końcowe zapytanie SQL w jednym przebiegu.

- ReAct - Strategia ReAct (Reason and Act) [9] łączy proces rozumowania z wykonywaniem akcji w środowisku zewnętrznym. Model językowy generuje kolejne kroki działania, wykonuje zapytania SQL lub operacje pomocnicze, analizuje wyniki ich wykonania oraz w razie potrzeby modyfikuje swoje decyzje. W implementacji systemu wykorzystany zostanie komponent SQL Agent frameworka LangChain [6], który domyślnie realizuje logikę działania zgodną z podejściem ReAct. Strategia ta stanowi zatem naturalny wariant bazowy dla systemów konwersacyjnych operujących na bazach danych.

Porównywane strategie tworzenia promptów self-correction

- Gentle self-correction prompt [5] - nie zakłada błędu, lecz prosi model o sprawdzenie zapytania i ewentualne wskazanie problemów, podając wskazówki dotyczące elementów SQL do weryfikacji.
- Generic prompt [5] - Zakłada, że zapytanie jest błędne i poleca modelowi zidentyfikować oraz poprawić błędy.

Obie strategie zostały zaimplementowane w strategii zero-shot, czyli bez dodatkowego trenowania modelu na danych specyficznych dla zadania.

Wybrane modele LLM

Porównamy ze sobą modele o otwartych wagach dostępne na platformie HuggingFace. Ograniczymy się do mniejszych modeli, możliwych do uruchomienia lokalnie lub na darmowych platformach. Wstępnie planujemy porównać model z rodziny Llama [10] oraz Bielika.

Przegląd

Zagadnienie wykorzystania modeli językowych do zastosowań z zakresu analizy danych i zadań BI dla baz relacyjnych jest szeroko badane. Typowe rozwiązania obejmują podejścia typu text-to-sql, a także text-to-python i text-to-dsl jako popularne alternatywy do obrabiania ustrukturyzowanych danych [11].

W celu zrozumienia struktury tabeli, typowo dane tabelaryczne są serializowane do jakiejś postaci tekstowej (markdown, json, itp.) co pozwala na bezpośrednie umieszczenie danych w prompcie, natomiast wyzwaniem jest ograniczony rozmiar kontekstu. Badane są także alternatywne podejścia, np. przetwarzanie obrazu tabeli modelami typu VLM, przetwarzanie tabeli w postaci struktur grafowych, a także szczególnie interesujące: traktowanie danych tabelarycznych jako odrębnej modalności. Przykładem takiego rozwiązania jest TableGPT2 [12], które łączy koder danych tabelarycznych, produkujący semantyczne osadzenia dla kolumn tabeli, połączony z modelem językowym typu tylko-dekoder wykorzystującym te osadzenia oraz prompt użytkownika.

Sposobem na rozwiązanie problemu z rozmiarem kontekstu, wzbogacenie promptu użytkownika o wiedzę domenową, a także wydobywanie informacji o schemacie bazy danych, stosowane są metody znane z zastosowań typu RAG, zadania *retrieval*.

Często stosowane zbiory benchmarkowe oraz typowe miary oceny jakości zostały opisane we wcześniejszych punktach.

Bibliografia

- [1] T. Yu *i in.*, „Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. 2018. Dostępne na: <https://arxiv.org/abs/1809.08887>
- [2] V. Zhong, C. Xiong, i R. Socher, „Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning”. 2017. Dostępne na: <https://arxiv.org/abs/1709.00103>
- [3] J. Li, B. Hui, *i in.*, „Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs”. 2023. Dostępne na: <https://arxiv.org/abs/2305.03111>
- [4] Y. Shi i et al. Tang, „A Survey on Employing Large Language Models for Text-to-SQL Tasks”. 2024. Dostępne na: <https://arxiv.org/abs/2407.15186>
- [5] M. Pourreza i D. Rafiei, „DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction”. 2023. Dostępne na: <https://arxiv.org/abs/2304.11015>
- [6] LangChain, „Build a SQL agent”. [Online]. Dostępne na: <https://docs.langchain.com/oss/python/langchain/sql-agent#run-your-agent-in-studio>
- [7] LangChain, „Build a custom SQL agent”. [Online]. Dostępne na: <https://docs.langchain.com/oss/python/langgraph/sql-agent>
- [8] L. Wang *i in.*, „Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models”. 2023. Dostępne na: <https://arxiv.org/abs/2305.04091>
- [9] S. Yao, J. Zhao, *i in.*, „ReAct: Synergizing Reasoning and Acting in Language Models”. 2022. Dostępne na: <https://arxiv.org/abs/2210.03629>
- [10] H. Touvron *i in.*, „Llama 3: Open Foundation and Instruction Models”. 2024. Dostępne na: <https://arxiv.org/abs/2302.13971>
- [11] J. Tian *i in.*, „Toward Real-World Table Agents: Capabilities, Workflows, and Design Principles for LLM-based Table Intelligence”. 2025. Dostępne na: <https://arxiv.org/pdf/2507.10281>
- [12] A. Su *i in.*, „TableGPT2: A Large Multimodal Model with Tabular Data Integration”. 2024. Dostępne na: <https://arxiv.org/abs/2411.02059>